

ViT-Ride: A Hybrid Vision Transformer Framework for Enhancing Two-Wheeler Safety via Smart Ignition Control

SWE3004 – Software Design and Development Project

Submitted in partial fulfillment of the requirements for the degree of

M.Tech (Software Engineering)

in

Department of Software and Systems Engineering

By

Muthukumar S

21MIS0272

Under the guidance of

Dr. Sivashankari R

Assistant Professor Sr. Grade 2

School of Computer Science Engineering and Information Systems

VIT, Vellore



November, 2025

ViT-Ride: A Hybrid Vision Transformer Framework for Enhancing Two-Wheeler Safety via Smart Ignition Control

SWE3004 - Software Design and Development Project

Submitted in partial fulfillment of the requirements for the degree of

M.Tech (Software Engineering)

in

Department of Software and Systems Engineering

By

Muthukumar s
21MIS0272

Under the guidance of

Dr. Sivashankari R
Assistant Professor Sr. Grade 2

School of Computer Science Engineering and Information Systems

VIT, Vellore



November, 2025

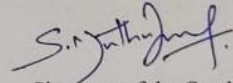
DECLARATION

I hereby declare that the SWE3004 – Software Design and Development Project entitled “ViT-Ride: A Hybrid Vision Transformer Framework for Enhancing Two-Wheeler Safety via Smart Ignition Control ” submitted by me, for the award of the degree of *M.Tech(Software Engineering) in Department of Software and Systems Engineering, School of Computer Science Engineering and Information Systems* to VIT is a record of bonafide work carried out by me under the supervision of **Dr. Sivashankari R , Assistant Professor Sr. Grade 2,SCORE, VIT, Vellore.**

I further declare that the work reported in this Project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree ordiploma in this institute or any other institute or university.

Place: Vellore

Date: 18/11/25



Signature of the Candidate

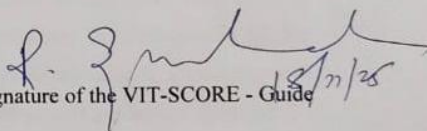
CERTIFICATE

This is to certify that the SWE3004 – Software Design and Development Project entitled “ViT-Ride: A Hybrid Vision Transformer Framework for Enhancing Two-Wheeler Safety via Smart Ignition Control” submitted by MUTHUKUMAR S 21MIS272, SCORE, VIT, for the award of the degree of *M.Tech (Software Engineering) in Department of Software and Systems Engineering*, is a record of bonafide work carried out by him under my supervision during the period, 9.7.2025 to 14.11.2025, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfill the requirements and regulations of the University and in my opinion meet the necessary standards for submission.

Place: Vellore

Date: 18/11/25


Signature of the VIT-SCORE - Guide

Internal Examiner

External Examiner

Head of the Department
Department of Software and Systems Engineering

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my SWE3004 – Software Design and Development Project guide **Dr. Sivashankari R, Assistant Professor Sr. Grade 2,** School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for his constant guidance, continual encouragement, in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of Domain **Deep Learning / Embedded Systems.**

"I would like to express my heartfelt gratitude to Honorable Chancellor Dr. G Viswanathan; respected Vice Presidents Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Vice Chancellor Dr. V. S. Kanchana Bhaaskaran; Pro-Vice Chancellor Dr. Partha Sharathi Mallick; and Registrar Dr. Jayabarathi T.

My whole-hearted thanks to Dean Dr. Daphne Lopez, School of Computer Science Engineering and Information Systems, Head, Department of Software and Systems Engineering Dr. Thanapal P, M.Tech (SE) Project Coordinators Dr. Thilagavathi M, Dr. Brijendra Singh, SCORE School Project Coordinator Dr. Thandeewaran R, all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my Project successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the project.

Place: Vellore

Date: 18/11/25

MUTHUKUMAR S

Executive Summary

Road accidents caused by helmet negligence and drunk driving remain a critical safety challenge, especially in countries with high two-wheeler usage. Existing solutions are mostly passive, relying on visual inspections or manual enforcement, and fail to prevent unsafe vehicle operation in real time. To address this gap, this project presents an integrated Real-Time Helmet Detection and Alcohol Monitoring System with Smart Ignition Control, combining deep learning-based computer vision with IoT sensing and embedded automation.

The system employs a Hybrid Vision Transformer (ViT) + CNN model to classify live webcam frames into Helmet or No Helmet categories. The fusion of global transformer attention and local CNN feature extraction ensures superior accuracy in diverse lighting and orientation conditions. Simultaneously, an MQ-3 gas sensor connected to an ESP32 microcontroller continuously analyzes breath samples to detect alcohol levels. Sensor readings are processed in real time, and safety decisions are communicated through serial-based control signals.

A core contribution of this system is its automatic ignition enforcement logic, which activates the vehicle only when both safety conditions are satisfied: Helmet = Yes and Alcohol = Safe. If either condition is violated, the system prevents ignition or initiates a 60-second safety shutdown sequence using OLED display feedback, buzzer alerts, and relay control to ensure user compliance.

The system also provides live monitoring, visual feedback, and event logging to support usability and safety audits. Experimental results demonstrate a helmet detection accuracy of 99.88%, fast inference performance, and reliable alcohol sensing, confirming the practical feasibility of the solution for real-world deployment.

By combining deep learning, embedded electronics, and automated ignition control, this project delivers an end-to-end, proactive safety enforcement mechanism aimed at reducing head injury fatalities and alcohol-related crashes. The architecture is scalable for future extensions, including GPS tracking, cloud connectivity, mobile app alerts, and multi-sensor rider safety analytics—making it a valuable framework for smart transportation, rider safety, and intelligent vehicle control systems.

CONTENTS	Page No.
Acknowledgement	v
Executive Summary	Vi
Table of Contents	Vii
List of Figures	ix
List of Tables	ix
Abbreviations	x
Symbols and Notations	xi
1 INTRODUCTION	1
1.1 Objective	2
1.2 Motivation	2
1.3 Background	2
1.4 Problem Statement	3
2 DISSERTATION / INTERNSHIP DESCRIPTION AND GOALS	4
2.1 Description	4
2.2 Goals	5
3 TECHNICAL SPECIFICATION	6
3.1 Hardware Requirements	6
3.2 Software Requirements	6
3.3 Libraries and Frameworks	6
3.4 Dataset Specifications	7
3.5 Model Architecture	9

4	DESIGN APPROACH AND DETAILS (as applicable)	10
4.1	Design Approach / Materials & Methods	10
4.2	Codes and Standards	14
4.3	Constraints, Alternatives and Tradeoffs	15
5	SCHEDULE, TASKS AND MILESTONES	17
6	DISSERTATION / INTERNSHIP DEMONSTRATION	19
6.1	Sample Codes	20
6.2	Sample Screen Shots	43
7	COST ANALYSIS / RESULT & DISCUSSION (as applicable)	50
8	SUMMARY	52
9	REFERENCES	54

List of Figures

Figure No.	Title	Page No.
Figure 1	Dataset Structure	7
Figure 2	Data Augmentation Samples	8
Figure 3	Data Flow Diagram	12
Figure 4	System Architecture Diagram	13
Figure 5	Hardware Setup	43
Figure 6	Helmet Detected (Safe Condition)	43
Figure 7	No Helmet Detected	44
Figure 8	Helmet Detected + Alcohol Safe (Motor ON)	45
Figure 9	No Helmet + Alcohol Safe	45
Figure 10	Alcohol Detected (Unsafe Condition)	46
Figure 11	Both Conditions Satisfied (Safe Operation)	46
Figure 12	60-Second Shutdown Countdown Display	47
Figure 13	Model Accuracy vs Epochs	48
Figure 14	Model Loss vs Epochs	48
Figure 15	Learning Curve – Accuracy	49
Figure 16	Learning Curve – Loss	49

List of Table

Table No.	Title	Page No.
1	PERFORMANCE METRICS SUMMARY	47
2	CLASS-WISE PERFORMANCE REPORT	47
3	COST ANALYSIS	50

List of Abbreviations

Abbreviation Full Form

AND	Logical AND Condition
BAC	Blood Alcohol Concentration
CNN	Convolutional Neural Network
DL	Deep Learning
ESP32	Espressif Systems 32-bit Microcontroller
GPU	Graphics Processing Unit
IoT	Internet of Things
LCD	Liquid Crystal Display
MQ-3	Alcohol Gas Sensor
OLED	Organic Light Emitting Diode
PC	Personal Computer
RAM	Random Access Memory
RMSE	Root Mean Square Error
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
ViT	Vision Transformer

Symbols and Notations

Symbol /Notation	Meaning
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
ACC	Accuracy
P	Precision
R	Recall
F1	F1-Score
MAE	Mean Absolute Error
RMSE	Root Mean Square Error
—	Division / Ratio Expression
×	Multiplication
%	Percentage
→	Output or mapping notation
=	Equality
Σ	Summation
$\hat{y}$	Predicted value
y	Actual / True value

CHAPTER 1

INTRODUCTION

Two-wheeler transportation is widely used across developing and densely populated countries due to its affordability, convenience, and accessibility. However, this mode of transport is also linked to a high rate of severe road accidents, often caused by two primary factors: the absence of helmet usage and drunk driving. According to global and national safety reports, a significant portion of motorcycle-related deaths could have been prevented if helmets were consistently worn and alcohol consumption was avoided. While traffic regulations exist to enforce these rules, manual enforcement has proven insufficient due to limited resources, human error, and non-compliance by riders. Therefore, a reliable, automated, and intelligent safety mechanism is required to address these challenges.

Recent advancements in deep learning and embedded IoT platforms have opened new opportunities to create smart systems capable of real-time safety enforcement without human intervention. Vision Transformers (ViT), which have revolutionized computer vision tasks through self-attention mechanisms, outperform traditional CNN-based approaches due to their ability to capture both global and local image dependencies. When combined with lightweight convolutional layers, Hybrid ViT models achieve faster inference and improved robustness, making them suitable for real-time helmet detection.

Parallel to visual analysis, alcohol detection systems using sensors like the MQ-3 can measure ethanol levels from the rider's breath. When integrated with a microcontroller such as the ESP32, this sensor can continuously monitor alcohol presence and trigger actions based on predefined safety thresholds. By combining helmet detection and alcohol sensing with ignition control logic, the system ensures that the vehicle operates only when both conditions are safe. This project presents an integrated real-time safety solution where the Hybrid ViT model performs helmet recognition on video input, while the ESP32 processes MQ-3 sensor data and controls ignition accordingly. The goal is to reduce preventable road injuries by automating compliance and providing timely alerts to encourage responsible riding behavior.

1.1 Objective

The objective of this project is to design and implement an intelligent safety system that prevents two-wheeler ignition unless the rider is wearing a helmet and is free from alcohol influence. By integrating a Hybrid Vision Transformer (ViT) model for real-time helmet detection with an MQ-3 alcohol sensor controlled by an ESP32 microcontroller, the system ensures automated and reliable safety enforcement. The goal is to eliminate human dependency in monitoring road safety rules and reduce accident rates caused by negligence and drunk riding. This project aims to provide a low-cost, scalable, and real-time solution suitable for modern smart transportation systems.

1.2 Motivation

The motivation behind this work arises from the increasing number of road fatalities due to riders not wearing helmets and operating vehicles while intoxicated. Traditional enforcement methods such as traffic checks and awareness campaigns are often inconsistent, ineffective, and resource-dependent. With rapid advancements in deep learning and embedded systems, there is a strong need to develop automated safety solutions that work proactively. This project is driven by the desire to reduce preventable road accidents, encourage responsible riding habits, and create a self-regulating system that ensures compliance without human intervention. Ultimately, it aims to improve road safety and save lives through intelligent technology integration.

1.3 Background

Recent advancements in intelligent transportation systems have accelerated the adoption of Vision Transformers (ViTs) and IoT-based safety control solutions. Traditional helmet detection methods rely heavily on Convolutional Neural Networks (CNNs), which are effective for local feature extraction but often struggle with complex backgrounds and varying lighting conditions. In contrast, ViTs offer superior global attention and context-aware representation, making them more robust for real-time helmet recognition. Similarly, MQ-3 sensors are widely used for alcohol detection due to their sensitivity, but most implementations only provide alerts rather than automated ignition control. Integrating ViT-based vision with IoT-driven sensor logic enables a fully autonomous, safety-enforced ignition system.

1.4 Problem Statement

Despite the existence of traffic laws and safety awareness campaigns, a large number of two-wheeler riders continue to ride without helmets or under the influence of alcohol, leading to thousands of avoidable accidents and fatalities each year. The major limitation lies in the lack of real-time, automated enforcement mechanisms. Most available systems are reactive, functioning only after an incident occurs or providing passive alerts that depend on user compliance. Traditional helmet detection approaches based on conventional CNN models often struggle in uncontrolled environments, while standalone alcohol detection systems using MQ-3 sensors typically generate warnings but do not prevent ignition. There is no unified framework that combines deep learning-based visual intelligence with IoT-driven sensor data to enforce safety before the vehicle starts. As a result, unsafe riding behavior continues unchecked. Therefore, a hybrid safety system capable of simultaneously detecting helmet usage and alcohol presence and automatically controlling ignition is urgently required to minimize human error and enhance road safety.

CHAPTER 2

DISSERTATION DESCRIPTION AND GOALS

2.1 Description

The proposed system introduces an intelligent, automated safety mechanism designed specifically for two-wheeler riders. It integrates advanced computer vision and IoT-based sensing to eliminate unsafe riding behaviors, such as operating a vehicle without wearing a helmet or driving under the influence of alcohol. The core of the system is a Hybrid Vision Transformer (ViT) model, which combines convolutional neural networks (CNNs) for local feature extraction with transformer-based self-attention layers for high-level contextual understanding. This architecture makes the model highly effective in identifying helmet usage under varying lighting conditions, backgrounds, camera angles, and partial occlusions, far outperforming traditional CNN-only methods.

Parallel to the computer vision module, the system utilizes an MQ-3 alcohol sensor, interfaced with an ESP32 microcontroller, to detect ethanol concentration in the rider's breath. Instead of calculating Blood Alcohol Concentration (BAC), the analog sensor output is compared against a calibrated safety threshold. If the predicted level exceeds the threshold, the system classifies it as unsafe. All decisions are handled in real time by the ESP32, which serves as the central controller that fuses helmet detection output from the PC with alcohol data from the sensor.

The system features a user-friendly interface that displays live status messages on an OLED screen, accompanied by audible and visual warnings through buzzer and LED indicators. For added safety, the system includes a smart shutdown mechanism, allowing the motor to continue for 60 seconds after an unsafe event is detected, giving the rider time to stop safely before ignition is disabled.

This integrated solution ensures that ignition is activated only when both safety conditions—helmet detection and alcohol-free status—are satisfied. By combining real-time AI inference with low-cost hardware, the system provides a reliable and scalable solution for improving on-road safety and reducing accident-related casualties.

2.2 Goals

The primary goal of this project is to create a fully automated, real-time rider safety enforcement system that eliminates human dependency in verifying helmet compliance and rider sobriety. By integrating computer vision and IoT-based sensing, the system aims to reduce road accidents and enforce safety regulations autonomously, making motorcycling significantly safer and more responsible.

One major goal is to achieve accurate and reliable helmet detection using a hybrid Vision Transformer architecture capable of handling real-world challenges such as changing lighting conditions, head orientation, occlusion, and motion blur. Through the fusion of CNN and Transformer components, the system aims to achieve high accuracy, robustness, and fast inference speeds suitable for live deployment.

Another goal is to detect alcohol consumption using the MQ-3 sensor and enforce ignition control without human intervention. Unlike basic alcohol-detection systems that only provide warnings, this system seeks to make ignition dependent on sensor output to prevent intoxicated riding in real-world conditions.

The system also aims to implement a smart failsafe mechanism instead of instant motor cut-off. A 60-second shutdown timer is introduced to allow the rider to safely stop the vehicle if helmet removal or alcohol detection happens mid-ride. This minimizes risks caused by abrupt power loss while ensuring safety compliance.

A further goal is to design a compact, low-cost, and easily deployable hardware framework using ESP32, buzzer, relay, OLED display, and basic sensors, enabling practical integration with two-wheelers without major modifications.

Lastly, the project aims to contribute to research in AI-based transportation safety, demonstrating that hybrid transformer architectures and IoT microcontrollers can work together to enforce behavioral compliance. By achieving this, the system represents a major step toward real-time intelligent road safety infrastructure, reducing human error and saving lives.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 Hardware Requirements

Recommended Hardware (Used in Implementation)

- Microcontroller: ESP32 Dev Board (Wi-Fi + ADC support)
- Sensor: MQ-3 (Alcohol Detection Sensor)
- Computer Vision Input: USB Webcam (720p or above)
- Display & Alerts: 0.96" OLED (SSD1306), Buzzer, LED Indicator
- Ignition/Motor Control: DC Motor / Relay Module
- Power Supply: 5V USB / Power Adapter
- Prototyping Components: Breadboard, Jumper Wires, USB Cable

Server / Model Training Machine (For Hybrid ViT Model)

1. Processor: DELL Ryzen 3
2. RAM: 8GB
3. GPU: NVIDIA GTX 1650 / RTX 2060 / Colab GPU (T4, L4, etc.)
4. Storage: Minimum 20 GB (Dataset + Model weights)
5. OS: Windows+ Python + PyTorch

3.2 Software Requirements

- Operating System: Windows 10 / Linux / macOS
- Development Tools:
 1. Arduino IDE (ESP32 Programming)
 2. Google Colab((Model Training)
 3. Visual Studio Code (Live detection code)
- Programming Languages: Python 3.9+, C/C++ for ESP32

3.3 Libraries and Frameworks

The system combines deep learning, image processing, and IoT control, requiring the use of several key libraries and frameworks:

3.3.1 Deep Learning Frameworks

- PyTorch – Core deep learning framework used for training and running the Hybrid ViT model
- timm (PyTorch Image Models) – Provides pretrained Vision Transformer architectures and utilities
- Torchvision – Dataset utilities, transforms, and image operations

3.3.2 Computer Vision & Image Processing

- OpenCV – Real-time image capture, preprocessing, and display
- Albumentations – Advanced image augmentation for improved model generalization
- PIL (Pillow) – Image handling and format conversion

3.3.3 Data Science & Evaluation

- NumPy – Matrix and tensor operations
- Pandas – Data handling and result logging
- Matplotlib / Seaborn – Graphs, accuracy/loss plots, heatmaps
- scikit-learn – Confusion matrix, classification report, performance metrics

3.3.4 ESP32 / Embedded Development

- Arduino Core for ESP32 – Firmware development in C/C++
- Wire / Adafruit_GFX / SSD1306 – OLED control libraries
- Serial / PySerial – Communication between Python and ESP32

3.4 Dataset Specifications:

The dataset used in this project was specifically created to train and evaluate the Hybrid Vision Transformer model for helmet detection in real-world conditions. It consists of a custom image dataset containing two classes: *Helmet* and *No Helmet*. The dataset includes approximately 4200 images captured from various sources including self-collected webcam images and publicly available internet datasets. Images were collected under diverse environments, lighting conditions, camera angles, and backgrounds to improve generalization capability.

```
Dataset Structure:
=====

TRAIN SET:
  Helmet: 1681 images
  No_helmet: 1680 images

VAL SET:
  Helmet: 421 images
  No_helmet: 420 images
```

Figure 1: Dataset Structure

Fig1: Shows how the dataset is organized into Helmet and No-Helmet folders for training and validation. This structure ensures the model can learn both classes properly during training.

For training the deep learning model, the dataset was split into 80% for training and 20% for validation, ensuring sufficient exposure during learning while maintaining unbiased evaluation. All raw images were standardized by resizing them to 224×224 pixels and converting them to JPG or PNG format for compatibility with the ViT architecture. Since the dataset follows a binary classification approach, annotations were provided at the image-level label format without bounding boxes or segmentation masks.

To improve robustness, various data augmentation techniques were applied including horizontal flipping, rotation, random brightness and contrast adjustments, Gaussian blur, and normalization using ImageNet statistics. These transformations simulate real-world variations such as head movement, partial occlusion, and inconsistent lighting. The dataset was designed to be nearly class-balanced to avoid model bias toward a particular category.

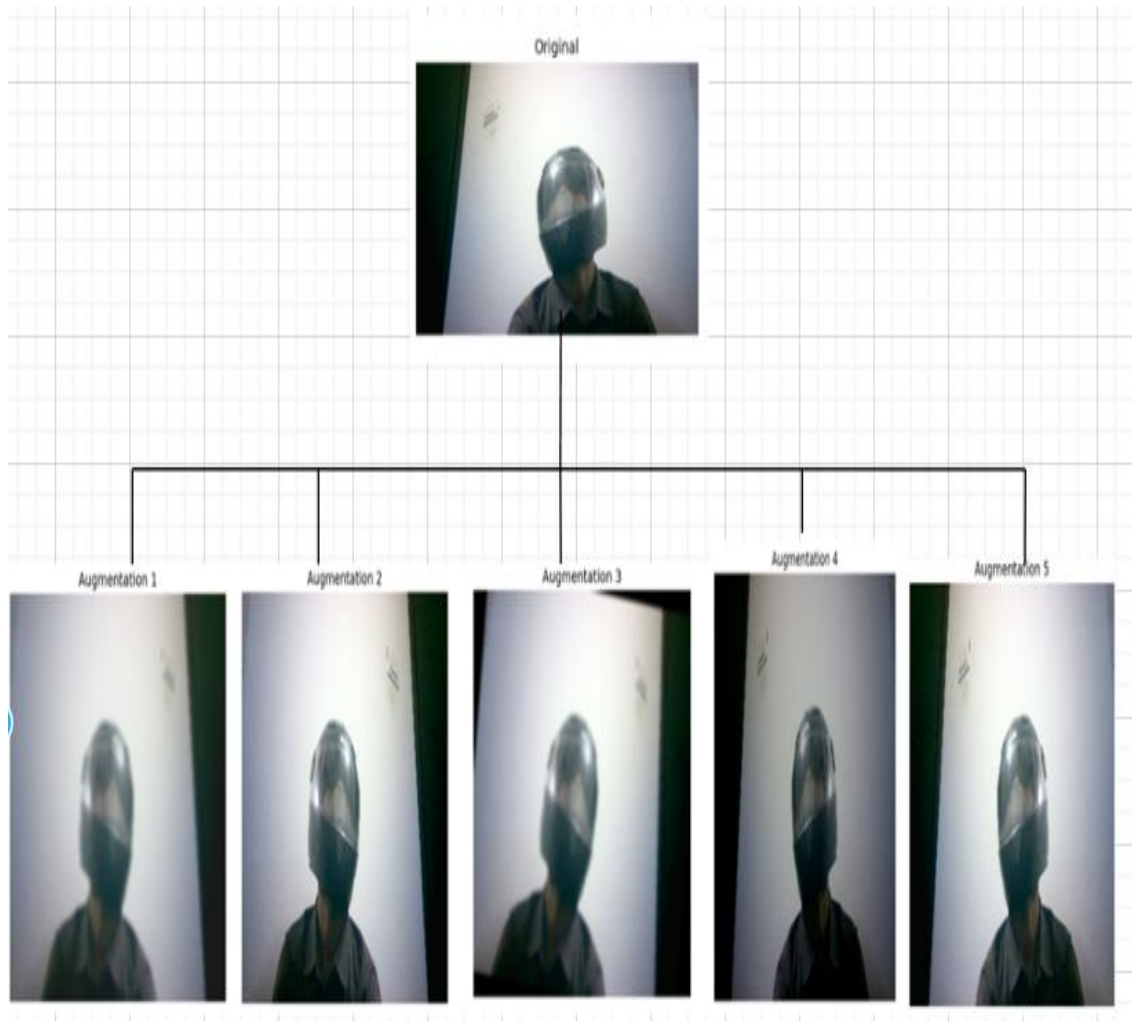


Figure 2: Data Augmentation

Fig:2 Displays examples of transformed images like rotation, flip, blur, etc. These augmentations improve model robustness in real-world conditions.

Overall, this dataset ensures a diverse, scalable, and realistic training environment that helps the model learn both global and local helmet-related visual features effectively, contributing to reliable performance in live deployment.

3.5 Model Architecture

- Model Type: Hybrid Vision Transformer (ViT) + CNN Fusion Model
- Backbone: ViT-Base (Patch Size 16×16 , 12 Transformer layers)
- CNN Branch: 3-layer convolutional feature extractor ($64 \rightarrow 128 \rightarrow 256$ filters)
- Fusion Layer: Feature concatenation ($768 + 256 = 1024$ dimensions)
- Output Classes: Helmet / No Helmet
- Framework: PyTorch + timm

3.5.1 Prediction Outputs

- Real-time helmet status detection (Helmet / No Helmet)
- Confidence score for each prediction
- Safety enforcement via ESP32 (ignition ON/OFF control)
- Can be extended for video-based continuous monitoring

CHAPTER 4

DESIGN APPROACH AND DETAILS

4. Design Approach

The proposed Smart Helmet Detection and Alcohol-Control System follows a hybrid, modular, and scalable architecture, enabling seamless integration between computer vision-based helmet detection and IoT-based alcohol monitoring. The system is designed to automate ignition control, ensuring that a vehicle starts only under safe riding conditions. The approach emphasizes:

- AI-driven helmet detection using a Hybrid Vision Transformer and CNN
- IoT-based alcohol sensing using the MQ-3 sensor and ESP32 microcontroller
- Serial communication for real-time interaction between AI model and hardware
- Confidence-based filtering to eliminate false helmet predictions
- Automatic motor control based on combined safety logic
- Real-time alerts through OLED display, buzzer, and LED indicators

✓ System workflow

The complete workflow of the system consists of the following stages:

✓ Image Acquisition

Live video frames are captured through a USB webcam and sent to the Hybrid ViT module.

✓ Preprocessing

- Resize to 224×224
- Normalization (ImageNet statistics)
- Tensor conversion
- Data augmentation during training (rotation, flip, brightness, blur)

✓ Hybrid ViT-CNN Model Inference

The deep learning model performs:

- Feature extraction using ViT (global) + CNN (local)
- Feature fusion (1024-dimensional vector)
- Binary classification: Helmet / No Helmet
- Confidence score generation
- Confidence threshold rejection (unsafe if < threshold)

✓ Alcohol Detection Module (ESP32 + MQ-3)

- Continuous analog sampling
 - Threshold-based classification: Safe / Unsafe
 - Noise filtering using sample averaging
 - Status transmitted internally to control logic
- ✓ Decision and Ignition Control
- Safety Rule: Helmet = Yes AND Alcohol Safe = Yes
 - Valid → Motor ON
 - Invalid → Motor OFF, warning activated
 - Countdown shutdown if unsafe during operation

✓ Alert & Feedback System

OLED UI displays:

- Helmet status
- Alcohol level status
- Motor state and safety warnings

Local alerts include:

- Buzzer beeps
- LED indication
- Shutdown timer notifications

✓ Evaluation

System performance is validated using:

- Accuracy, precision, recall & F1-score for helmet detection
- Real-time sensor test validation for alcohol detection
- Latency and safety response evaluation

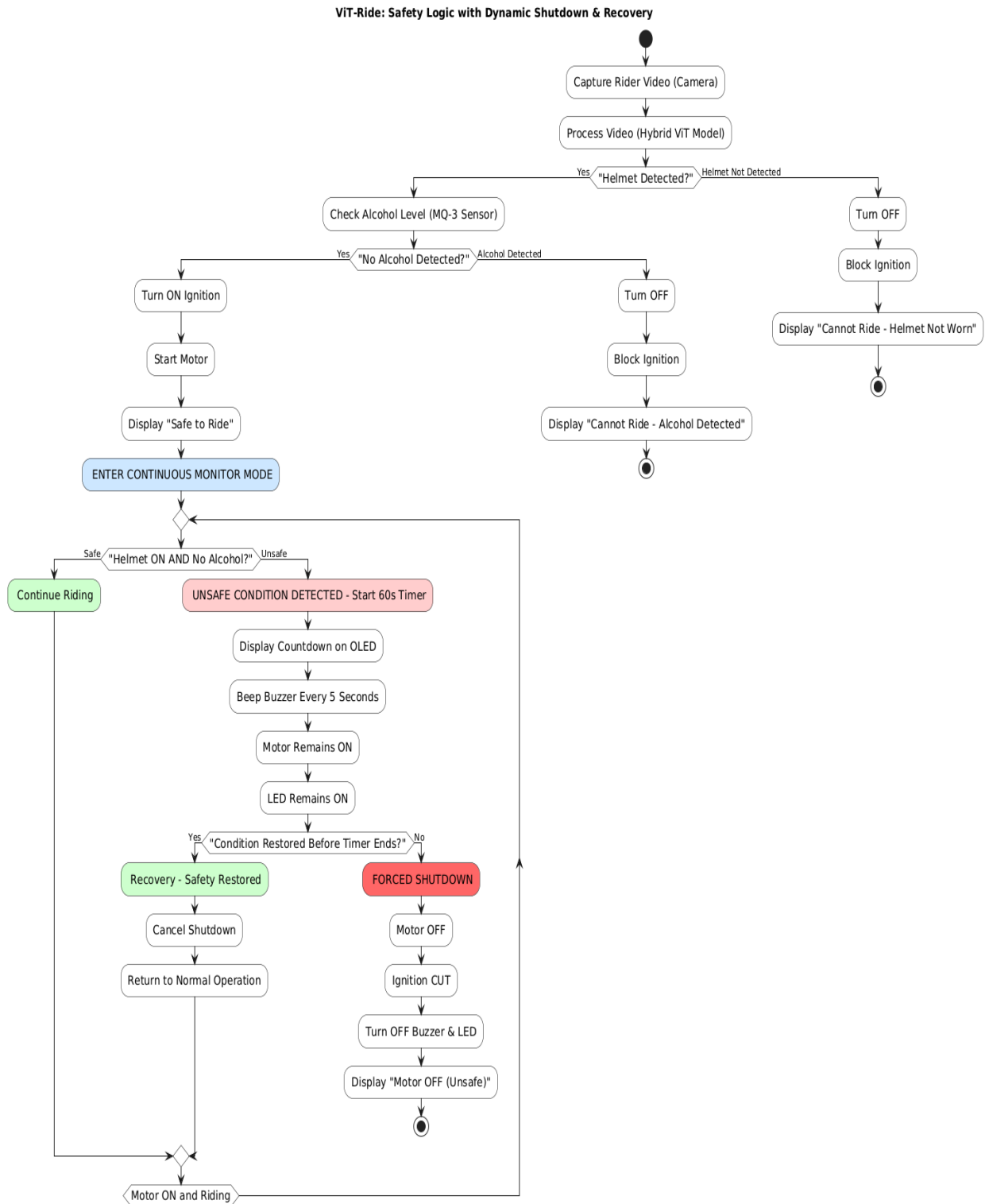


Figure 3: Data Flow Diagram

Fig: 3 Illustrates the step-by-step workflow from image capture to prediction and ignition control. It shows how helmet detection and alcohol sensing connect to the decision-making unit

✓ System Architecture

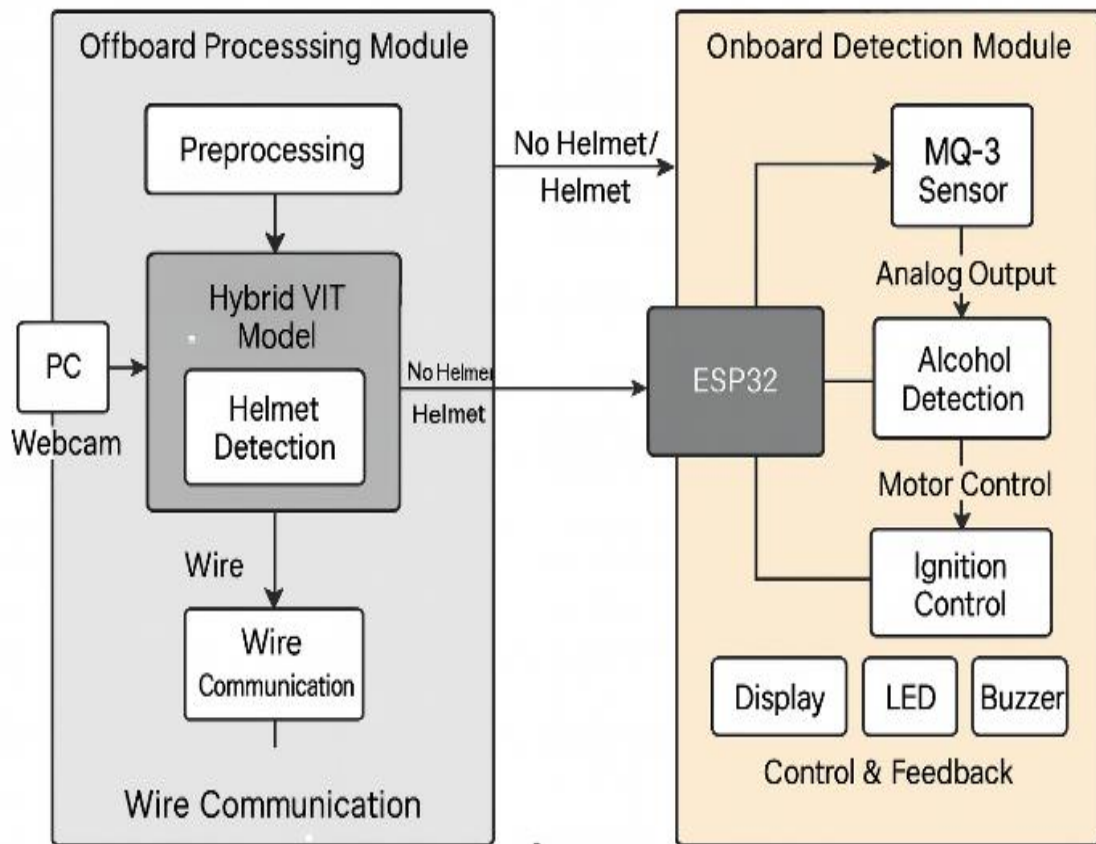


Figure 4: System Architecture Diagram

Fig 4 (System Architecture Diagram) represents the complete workflow of the Smart Helmet and Alcohol Detection System. It shows how the webcam sends live video frames to the Hybrid ViT-CNN deep learning model for helmet detection, while the MQ-3 alcohol sensor continuously measures breath alcohol levels through the ESP32 microcontroller. The outputs of both modules are combined using an AND logic to decide whether the motor/ignition should be enabled. The ESP32 also controls the OLED display, buzzer, and relay to provide real-time safety feedback and alerts.

Design Principles Followed

- Modularity: Vision pipeline, sensor module, and control logic operate independently
- Reusability: Model and microcontroller logic can be adapted for other safety use cases
- Scalability: Supports extension to cloud, mobile, GPS, and additional sensors
- Interpretability: Grad-CAM/attention maps and sensor thresholds provide transparency
- Safety-first Design: System defaults to motor OFF when uncertainty or unsafe conditions occur
- Performance Optimization: Lightweight inference, low-latency serial communication, and real-time control

4.2 Codes and Standards (Deep Learning + Embedded Systems Domain)

Although the system is mainly research-oriented, adherence to relevant engineering standards improves reproducibility, maintainability, and safety. The following standards and best practices were considered:

✓ Software & Deep Learning Standards

IEEE / ISO Standards

- IEEE 1016 – Software Design Documentation

Used for documenting system architecture, module interactions, and data flow.

- IEEE 829 / ISO/IEC/IEEE 29119 – Software Testing Documentation

Applied for test case development of:

- Helmet detection accuracy
- Sensor validation
- Decision logic & safety testing

- IEEE 12207 – Software Lifecycle Processes

Followed during planning, model training, deployment, and system maintenance.

✓ Deep Learning Practices

- Proper dataset split (train/validation/test)
- Normalization and preprocessing based on standardized methods (ImageNet stats)
- Reproducibility ensured using random seeds & deterministic behavior
- Model saved using PyTorch checkpoints and versioned via Git

- Evaluation metrics: Accuracy, Precision, Recall, F1-score
- Grad-CAM / Attention Maps used for explainability
- Use of PyTorch and timm libraries following standard ViT implementation guidelines

✓ Embedded System & IoT Standards

- UART communication uses reliable message formatting (ASCII protocol)
- Sensor handling follows ADC sampling best practices
- Electrical safety ensured using proper relay isolation
- Output signals controlled within safe voltage/current limits
- No personal data collected → No GDPR/privacy concerns
- MQTT/Wi-Fi alternatives considered for future IoT cloud integration

✓ Explainability & Responsible AI

Even though no formal global XAI standards exist, the project follows industry-accepted practices:

- Grad-CAM heatmap visualization
- ViT attention map interpretation
- Confidence thresholding for safety-based decisions
- System logs for debugging and bias detection

4.3 Constraints, Alternatives, and Tradeoffs

✓ Computational Constraints

- ViT inference requires a capable CPU/GPU

Alternative: Use MobileNet or TinyML

Tradeoff: Hybrid ViT retained for superior accuracy, inference done on PC instead of ESP32

✓ Embedded Hardware Limitations

- ESP32 cannot run deep learning inference due to limited RAM/flash

Alternative: Deploy TinyML model on MCU

Tradeoff: ESP32 kept only for sensing + ignition control while inference is offloaded to PC

✓ Sensor Constraints

- MQ-3 can produce noise and drift due to environmental factors

Alternative: Fuel cell or industrial-grade alcohol sensors

Tradeoff: MQ-3 selected for cost-effectiveness + availability

✓ Real-Time Constraints

- System must evaluate helmet + alcohol status before ignition is enabled

Alternative: Use batch processing or cloud inference

Tradeoff: Real-time local inference prioritized for safety

✓ Model Interpretability vs. Accuracy

- Deep learning models lack transparency compared to rule-based systems

Alternative: Classical image processing methods

Tradeoff: Hybrid ViT selected due to high accuracy; interpretability handled using Grad-CAM & attention maps

✓ Communication & Reliability Constraints

- UART communication may introduce latency or packet loss

Alternative: Wi-Fi, BLE, or MQTT-based protocol

Tradeoff: UART retained for simplicity, low-power use, and real-time performance

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

Phase No.	Project Phase / Task	Description of Work	Start Date	End Date	Milestone Achieved
1	Problem Definition & Literature Survey	Identify safety problems, review helmet + alcohol research, define objectives	Week 1	Week 2	Problem statement & literature review completed
2	Hardware Setup & Sensor Integration	Connect ESP32, MQ-3, relay, OLED, buzzer, LED; verify sensor readings	Week 2	Week 3	Hardware successfully integrated
3	Image Dataset Collection	Capture Helmet / No Helmet images via webcam and phone camera	Week 3	Week 4	Dataset collected(own dataset)
4	Dataset Preprocessing	Resize, normalize, augment images, label, split dataset	Week 4	Week 5	Dataset ready for model training
5	Hybrid ViT-CNN Model Design	Define architecture, configure transformer + CNN feature fusion	Week 5	Week 6	Model architecture finalized
6	Model Training	Train the hybrid model for 15–20 epochs and tune hyperparameters	Week 6	Week 8	Model training completed
7	Model Evaluation	Evaluate accuracy, precision, recall, F1-score, confusion matrix	Week 8	Week 9	Best model selected
8	ESP32 Firmware Development	Implement alcohol sensing, motor control, OLED, buzzer alerts	Week 9	Week 10	Firmware completed

9	Serial Communication Integration	Enable UART-based communication between AI model and ESP32	Week 10	Week 11	Real-time communication established
10	Safety Logic & Ignition Control	Implement AND logic for helmet + alcohol, relay-based motor control	Week 11	Week 12	Safe ignition logic functional
11	Live Detection – System Deployment	Real-time helmet detection + alcohol sensing + motor control	Week 12	Week 13	Live detection successfully demonstrated
12	System Optimization	Improve response time, tune thresholds, enhance detection reliability	Week 13	Week 14	Optimized real-time system
13	User Feedback Interface	OLED UI, LED indicators, buzzer alerts, shutdown timer	Week 14	Week 15	User feedback module completed
14	Final Documentation	Prepare full report (Ch 1–5, diagrams, screenshots, results)	Week 15	Week 16	Project documentation completed

CHAPTER 6

DISSERTATION DEMONSTRATION

The proposed system integrates deep learning-based vision analysis with embedded IoT sensing to deliver a fully automated safety enforcement mechanism for two-wheeler riders. It combines helmet detection using a Hybrid Vision Transformer (ViT + CNN) model with alcohol-level monitoring via an MQ-3 sensor connected to an ESP32 microcontroller. Live video frames from a webcam are preprocessed and classified as Helmet or No Helmet using the fusion-based architecture, where the ViT branch extracts global self-attention features and the CNN branch captures local spatial features, resulting in highly accurate predictions. The model is trained on a custom dataset with augmentations such as flipping, rotation, blur, and brightness variations, and evaluated using accuracy, precision, recall, F1-score, and confusion matrix to ensure real-world reliability. In parallel, the ESP32 continuously reads analog signals from the MQ-3 alcohol sensor and determines intoxication status while communicating with the AI model through serial communication. Ignition control is enforced using an AND condition where the motor relay is activated only if both helmet detection and alcohol safety are satisfied; otherwise, ignition remains disabled. To ensure rider safety while avoiding abrupt shutdown, a 60-second safety shutdown mechanism is implemented. When unsafe behavior is detected—such as helmet removal or alcohol detection during motion—the system enters an alert state, displays a countdown timer on the OLED screen, triggers buzzer beeps every five seconds, and allows the rider a 60-second correction window. If safety conditions are restored, shutdown is aborted and normal operation continues; if not, the system automatically turns off the motor relay, disables ignition, and displays a “Motor OFF – Unsafe” warning. The final system operates fully in real-time, combining AI-driven vision inference, sensor monitoring, and embedded control with multimodal feedback through OLED, LEDs, and buzzer alarms. This hybrid AI-IoT framework provides a proactive and intelligent solution to enforce helmet compliance and prevent alcohol-impaired riding, significantly reducing avoidable road accidents.

7.1 Sample Code

✓ (Hybrid ViT model Training code)

✓ CELL 1 — Mount Drive + Install Dependencies

```
from google.colab import drive
drive.mount('/content/drive')
```

```
!pip install timm alumentations opencv-python-headless
```

✓ CELL 2 — Imports + Config + Seed

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms
from torchvision.transforms import functional as F

import timm
import cv2
import numpy as np
import os
import matplotlib.pyplot as plt
from PIL import Image
from sklearn.metrics import confusion_matrix, classification_report
from tqdm import tqdm
import alumentations as A
from alumentations.pytorch import ToTensorV2
import random

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
```

```
torch.backends.cudnn.benchmark = False
```

```
set_seed(42)
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')  
print("Using device:", device)
```

✓ CELL 3 — Dataset Paths & Config

```
DATASET_PATH = '/content/drive/MyDrive/Datasplit'
```

```
CONFIG = {  
    'img_size': 224,  
    'batch_size': 16,  
    'num_epochs': 20,  
    'learning_rate': 1e-4,  
    'num_classes': 2,  
    'num_workers': 2,  
    'classes': ['Helmet', 'No_helmet']  
}
```

```
train_path = os.path.join(DATASET_PATH, 'train')  
val_path = os.path.join(DATASET_PATH, 'val')
```

✓ CELL 4 — Dataset Structure Print

```
for split in ['train', 'val']:  
    split_path = os.path.join(DATASET_PATH, split)  
    print(f"\n{split.upper()} SET:")  
  
    for class_name in CONFIG['classes']:  
        class_path = os.path.join(split_path, class_name)  
        if os.path.exists(class_path):  
            num_images = len([f for f in os.listdir(class_path)  
                              if f.lower().endswith(('.png', '.jpg', '.jpeg'))])  
            print(f" {class_name}: {num_images} images")
```


✓ CELL 5 — Dataset Class

```
class HelmetDataset(Dataset):  
    def __init__(self, root_dir, transform=None):  
        self.root_dir = root_dir  
        self.transform = transform  
        self.images = []  
        self.labels = []  
        self.class_to_idx = {'Helmet': 0, 'No_helmet': 1}  
  
        for class_name in self.class_to_idx.keys():  
            class_path = os.path.join(root_dir, class_name)  
            if not os.path.exists(class_path): continue  
  
            for img_name in os.listdir(class_path):  
                if img_name.lower().endswith(('.png', '.jpg', '.jpeg')):  
                    self.images.append(os.path.join(class_path, img_name))  
                    self.labels.append(self.class_to_idx[class_name])  
  
    def __len__(self):  
        return len(self.images)  
  
    def __getitem__(self, idx):  
        img = cv2.imread(self.images[idx])  
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  
        label = self.labels[idx]  
        if self.transform:  
            img = self.transform(image=img)['image']  
        return img, label
```

✓ CELL 6 — Transforms

```
train_transform = A.Compose([
    A.Resize(CONFIG['img_size'], CONFIG['img_size']),
    A.HorizontalFlip(p=0.5),
    A.Rotate(limit=15, p=0.5),
    A.RandomBrightnessContrast(p=0.5),
    A.GaussianBlur(blur_limit=5, p=0.3),
    A.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]),
    ToTensorV2()
])
```

```
val_transform = A.Compose([
    A.Resize(CONFIG['img_size'], CONFIG['img_size']),
    A.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]),
    ToTensorV2()
])
```

✓ CELL 7 — Load Datasets

```
train_dataset = HelmetDataset(train_path, transform=train_transform)
val_dataset = HelmetDataset(val_path, transform=val_transform)

train_loader = DataLoader(train_dataset, batch_size=CONFIG['batch_size'],
                           shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=CONFIG['batch_size'], shuffle=False)

print("Train batches:", len(train_loader))
print("Val batches:", len(val_loader))
```

✓ CELL 8 — Visualization Function

```
def show_samples(dataset, num=8):
    fig, axes = plt.subplots(2,4, figsize=(15,8))
    axes = axes.ravel()
    for i in range(num):
        idx = random.randint(0, len(dataset)-1)
```

```

img = cv2.imread(dataset.images[idx])
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
axes[i].imshow(img)
axes[i].set_title(CONFIG['classes'][dataset.labels[idx]])
axes[i].axis('off')
plt.show()

show_samples(train_dataset)

✓ CELL 9 — Hybrid ViT-CNN Model
class HybridViTModel(nn.Module):
    def __init__(self, num_classes=2, pretrained=True):
        super().__init__()
        self.vit = timm.create_model('vit_base_patch16_224', pretrained=pretrained)
        vit_dim = self.vit.embed_dim
        self.vit.head = nn.Identity()

        self.cnn_branch = nn.Sequential(
            nn.Conv2d(3,64,7,2,3),
            nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(3,2,1),
            nn.Conv2d(64,128,3,padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.MaxPool2d(2,2),
            nn.Conv2d(128,256,3,padding=1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1,1))
        )

        fusion_dim = vit_dim + 256
        self.fc = nn.Sequential(
            nn.Linear(fusion_dim,512), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(512,256), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(256,num_classes)
        )

    def forward(self,x):

```

```

vit_f = self.vit(x)
cnn_f = self.cnn_branch(x).flatten(1)
fused = torch.cat([vit_f, cnn_f], dim=1)
return self.fc(fused)

```

```

model = HybridViTModel(num_classes=2).to(device)

```

CELL 10 — Loss, Optimizer, Scheduler

```

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=CONFIG['learning_rate'],
weight_decay=0.01)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=CONFIG['num_epochs'])

```

✓ CELL 11 — Training & Validation Functions

```

def train_epoch(model, dataloader, criterion, optimizer, device):

```

```

    model.train()
    total_loss, correct, total = 0, 0, 0

```

```

    pbar = tqdm(dataloader, desc="Training")
    for images, labels in pbar:
        images, labels = images.to(device), labels.to(device)

```

```

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

```

```

        total_loss += loss.item()
        _, pred = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (pred == labels).sum().item()
    pbar.set_postfix({'loss': loss.item(), 'acc': 100*correct/total})

```

```
return total_loss / len(dataloader), 100 * correct / total
```

```
def validate_epoch(model, dataloader, criterion, device):
    model.eval()
    total_loss, correct, total = 0, 0, 0

    with torch.no_grad():
        pbar = tqdm(dataloader, desc="Validation")
        for images, labels in pbar:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            total_loss += loss.item()
            _, pred = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (pred == labels).sum().item()
            pbar.set_postfix({'loss': loss.item(), 'acc': 100*correct/total})

    return total_loss / len(dataloader), 100 * correct / total
```

✓ CELL 12 — Training Loop

```
history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}
best_val_acc = 0.0
best_model_path = '/content/drive/MyDrive/best_helmet_model.pth'

print("\n Training Started...\n")
for epoch in range(CONFIG['num_epochs']):
    print(f'Epoch {epoch+1}/{CONFIG['num_epochs']}')
    print("=====")
```

```

train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer, device)
val_loss, val_acc = validate_epoch(model, val_loader, criterion, device)

scheduler.step()

history['train_loss'].append(train_loss)
history['train_acc'].append(train_acc)
history['val_loss'].append(val_loss)
history['val_acc'].append(val_acc)

print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}%")
print(f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.2f}%")

if val_acc > best_val_acc:
    best_val_acc = val_acc
    torch.save({
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'val_acc': val_acc,
        'config': CONFIG
    }, best_model_path)
    print(f"Best model updated! Val Acc = {val_acc:.2f}%")

print("\n Training Complete!")
print(f"Best Validation Accuracy: {best_val_acc:.2f}%")

```

✓ CELL 13 — Load Best Model for Testing / Inference

```

import torch
import torch.nn as nn
import timm

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

```

```

class HybridViTModel(nn.Module):
    def __init__(self, num_classes=2, pretrained=True):
        super().__init__()
        self.vit = timm.create_model("vit_base_patch16_224", pretrained=pretrained)
        self.vit.head = nn.Identity()
        self.vit_dim = self.vit.embed_dim

        self.cnn = nn.Sequential(
            nn.Conv2d(3, 64, 7, 2, 3),
            nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(3, 2, 1),

            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128), nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(128, 256, 3, padding=1),
            nn.BatchNorm2d(256), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1,1))
        )

        self.fc = nn.Sequential(
            nn.Linear(self.vit_dim + 256, 512),
            nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(512, 256),
            nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        v = self.vit(x)
        c = self.cnn(x).flatten(1)
        return self.fc(torch.cat([v, c], dim=1))

```

```

CONFIG = {'num_classes': 2, 'classes': ['Helmet', 'No_helmet']}

model = HybridViTModel(num_classes=2, pretrained=False).to(device)

checkpoint = torch.load("/content/drive/MyDrive/best_helmet_model.pth",
map_location=device)
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()

print(f" Model Loaded (Best Val Acc: {checkpoint['val_acc']:.2f}%)")

```

✓ CELL 14 — Define Transform & Prediction Function

```

import cv2
import albumentations as A
from albumentations.pytorch import ToTensorV2
import matplotlib.pyplot as plt

test_transform = A.Compose([
    A.Resize(224,224),
    A.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]),
    ToTensorV2()
])

def predict_image(image_path):
    img = cv2.imread(image_path)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    transformed = test_transform(image=img_rgb)
    tensor = transformed['image'].unsqueeze(0).to(device)

    with torch.no_grad():
        output = model(tensor)
        prob = torch.softmax(output,1)

```



```

conf, pred = torch.max(prob,1)

label = CONFIG['classes'][pred.item()]
score = conf.item()*100
return img_rgb, label, score

```

✓ CELL 15 — Run Predictions on Folder Images

```
import os
```

```
test_folder = "/content/drive/MyDrive/test"
```

```

files = [f for f in os.listdir(test_folder) if f.lower().endswith(('jpg','jpeg','png'))]
print("Found test images:", len(files))

```

```
for f in files:
```

```

    path = os.path.join(test_folder, f)
    img, label, conf = predict_image(path)

```

```

plt.figure(figsize=(6,4))
plt.imshow(img)
plt.title(f'{label} ( {conf:.2f}%)', color=('green' if label=='Helmet' else 'red'))
plt.axis('off')
plt.show()
print(f'{f}: {label} ( {conf:.2f}%) \n")

```

✓ Live detection code

Requirements:

```
pip install torch torchvision timm opencv-python pillow numpy matplotlib pyserial
```

Usage:

```
python smart_helmet.py --mode webcam
```

```
"""
```

```
import torch
import torch.nn as nn
import cv2
import timm
from torchvision import transforms
import time
import os
import argparse
import serial
import numpy as np
from pathlib import Path
CONFIG = {
    'img_size': 224,
    'num_classes': 2,
    'classes': ['Helmet', 'No_helmet'],
    'model_path': r'C:\Users\DELL\Downloads\best_helmet_model.pth',
    'confidence_threshold': 0.90, # FIXED → stricter safety
    'capture_interval': 5,
    'serial_port': 'COM3',
    'baud_rate': 115200
}
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

esp32 = None
try:
```

```

esp32 = serial.Serial(CONFIG['serial_port'], CONFIG['baud_rate'], timeout=1)
print(f'Connected to ESP32 on {CONFIG['serial_port']}")
except Exception as e:
    print(f'Could not connect to ESP32: {e}")

```

```

class HybridViTModel(nn.Module):
    def __init__(self, num_classes=2, pretrained=False):
        super(HybridViTModel, self).__init__()
        self.vit = timm.create_model('vit_base_patch16_224', pretrained=pretrained)
        vit_embed_dim = self.vit.embed_dim

        self.cnn_branch = nn.Sequential(
            nn.Conv2d(3, 64, 7, stride=2, padding=3),
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            nn.MaxPool2d(3, stride=2, padding=1),
            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(128, 256, 3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(True),
            nn.AdaptiveAvgPool2d((1, 1))
        )

        self.vit.head = nn.Identity()
        fusion_dim = vit_embed_dim + 256
        self.fusion = nn.Sequential(
            nn.Linear(fusion_dim, 512),
            nn.ReLU(True),
            nn.Dropout(0.3),
            nn.Linear(512, 256),
            nn.ReLU(True),

```

```

        nn.Dropout(0.2),
        nn.Linear(256, num_classes)
    )

def forward(self, x):
    vit_features = self.vit(x)
    cnn_features = self.cnn_branch(x).flatten(1)
    combined = torch.cat([vit_features, cnn_features], dim=1)
    return self.fusion(combined)

def load_model():
    print("Loading model...")
    model = HybridViTModel(num_classes=CONFIG['num_classes'], pretrained=False)
    checkpoint = torch.load(CONFIG['model_path'], map_location=device)
    model.load_state_dict(checkpoint['model_state_dict'])
    print("Model loaded successfully!")
    model.to(device)
    model.eval()
    return model

transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((CONFIG['img_size'], CONFIG['img_size'])),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

def predict_frame(model, frame):
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    img_tensor = transform(frame_rgb).unsqueeze(0).to(device)
    with torch.no_grad():
        outputs = model(img_tensor)
        probs = torch.softmax(outputs, dim=1)

```

```

confidence, predicted = torch.max(probs, 1)

label = CONFIG['classes'][predicted.item()]
conf = confidence.item()

# FIX ADDED HERE: enforce No_helmet if low confidence
if label == "Helmet" and conf < CONFIG['confidence_threshold']:
    label = "No_helmet"

return label, conf

def send_to_esp32(label):
    if esp32 is None:
        return
    try:
        if label == "Helmet":
            esp32.write(b"HELMET:1\n")
        else:
            esp32.write(b"HELMET:0\n")
    except:
        pass

def draw_prediction(frame, label, confidence, fps=None):
    color = (0, 255, 0) if label == 'Helmet' else (0, 0, 255)
    status = "SAFE ✓" if label == 'Helmet' else "WARNING ⚠"

    cv2.rectangle(frame, (10, 10), (frame.shape[1]-10, frame.shape[0]-10), color, 3)
    cv2.putText(frame, f"{status}: {label}", (30, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1.0, color, 3)

    cv2.putText(frame, f"Conf: {confidence*100:.1f}%", (30, 75),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255,255,255), 2)

```

```

if fps:
    cv2.putText(frame, f"FPS: {fps}", (frame.shape[1]-150, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255,255,255), 2)

return frame

def webcam_mode(model):
    print("Starting Smart Helmet Detection...")
    cap = cv2.VideoCapture(0)

    prev_time = time.time()
    last_send_time = time.time()

    try:
        while True:
            ret, frame = cap.read()
            if not ret:
                break

            label, conf = predict_frame(model, frame)

            if time.time() - last_send_time > 1:
                send_to_esp32(label)
                last_send_time = time.time()

            curr_time = time.time()
            fps = int(1 / (curr_time - prev_time))
            prev_time = curr_time

            frame = draw_prediction(frame, label, conf, fps)
            cv2.imshow("Smart Helmet Detection", frame)

            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

```

```
finally:
    if esp32:
        esp32.write(b"STOP\n")
        time.sleep(0.3)
        esp32.close()

    cap.release()
    cv2.destroyAllWindows()
    print("System stopped safely.")
```

```
def main():
    parser = argparse.ArgumentParser(description='Smart Helmet Detection System')
    parser.add_argument('--mode', type=str, default='webcam')
    args = parser.parse_args()

    model = load_model()
    webcam_mode(model)

if __name__ == "__main__":
    main()
```

✓ Hardware code

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// === OLED Settings ===
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// === Pin Definitions ===
const int mq3Pin = 34;    // MQ-3 Analog input
const int buzzerPin = 26; // Buzzer output
const int ledPin = 25;    // LED output
const int motorPin = 27;  // Motor relay control

// === Alcohol Threshold ===
const int threshold = 1700;

// === Variables ===
bool helmetDetected = false;
bool alcoholSafe = false;
bool motorRunning = false;
bool shutdownInitiated = false;

unsigned long shutdownStartTime = 0;
const unsigned long shutdownDelay = 60 * 1000; // 1 minute
unsigned long lastBeepTime = 0;

void setup() {
  Serial.begin(115200);

  pinMode(buzzerPin, OUTPUT);
```



```

pinMode(ledPin, OUTPUT);
pinMode(motorPin, OUTPUT);
digitalWrite(motorPin, LOW);

// OLED Initialization
if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED init failed");
    for(;;);
}

display.clearDisplay();
display.setTextSize(2);
display.setTextColor(WHITE);
display.setCursor(10, 20);
display.println("SMART HELMET");
display.display();
delay(1500);
}

void showStoppedScreen() {
    digitalWrite(motorPin, LOW);
    digitalWrite(ledPin, LOW);
    digitalWrite(buzzerPin, LOW);

    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(WHITE);
    display.setCursor(20, 20);
    display.println("STOPPED");
    display.display();

    Serial.println(" System Stopped (Python closed)");
}

```

```

void loop() {

    // --- Receive Data from Python / PC ---
    if (Serial.available() > 0) {
        String data = Serial.readStringUntil('\n');
        data.trim();

        // --- STOP Command ---
        if (data == "STOP") {
            helmetDetected = false;
            alcoholSafe = false;
            motorRunning = false;
            shutdownInitiated = false;

            showStoppedScreen();
            delay(1000);
            return; // stop loop actions
        }

        // --- Helmet Detection ---
        if (data.startsWith("HELMET:")) {
            helmetDetected = (data.substring(7).toInt() == 1);
            Serial.print("Helmet status: ");
            Serial.println(helmetDetected ? "Detected" : "Not Detected");
        }
    }

    // --- Alcohol Sensor Read ---
    int sensorValue = analogRead(mq3Pin);
    alcoholSafe = (sensorValue < threshold);

    // --- Display Info ---
    display.clearDisplay();
    display.setTextSize(1);

```

```

display.setTextColor(WHITE);

display.setCursor(0, 0);
display.print("Helmet: ");
display.println(helmetDetected ? "Yes" : "No");

display.setCursor(0, 12);
display.print("Alcohol: ");
display.println(alcoholSafe ? "Safe" : "Detected");

unsigned long now = millis();

// === SAFE CONDITION ===
if (helmetDetected && alcoholSafe) {
    digitalWrite(motorPin, HIGH);
    digitalWrite(ledPin, HIGH);
    digitalWrite(buzzerPin, LOW);

    if (shutdownInitiated) {
        shutdownInitiated = false;
        Serial.println("Safe again — shutdown cancelled.");
        display.setCursor(0, 35);
        display.println("Safe again!");
        display.setCursor(0, 47);
        display.println("Motor continues");
        display.display();
        delay(1000);
    }

    motorRunning = true;
    display.setCursor(0, 35);
    display.println("Motor: ON (Safe)");
}

```

```

// === UNSAFE WHILE MOTOR RUNNING ===
else if (motorRunning) {

    if (!shutdownInitiated) {
        shutdownStartTime = now;
        shutdownInitiated = true;
        Serial.println("Unsafe! Motor will off in 1 minute.");

        digitalWrite(buzzerPin, HIGH);
        display.setCursor(0, 35);
        display.println("Unsafe!");
        display.setCursor(0, 47);
        display.println("Motor off in 60s");
        display.display();
        delay(1000);
        digitalWrite(buzzerPin, LOW);
    }

    else {
        unsigned long elapsed = now - shutdownStartTime;
        unsigned long remaining = (shutdownDelay - elapsed) / 1000;

        display.setCursor(0, 35);
        display.print("Unsafe! Off in ");
        display.print(remaining);
        display.println("s");

        if (remaining % 5 == 0 && now - lastBeepTime > 1000) {
            digitalWrite(buzzerPin, HIGH);
            delay(200);
            digitalWrite(buzzerPin, LOW);
            lastBeepTime = now;
        }
    }
}

```

```

if (helmetDetected && alcoholSafe) {
    shutdownInitiated = false;
    Serial.println("Recovered — motor continues.");
    display.setCursor(0, 35);
    display.println("Safe again!");
    display.setCursor(0, 47);
    display.println("Motor continues");
    display.display();
    delay(1000);
    motorRunning = true;
}
if (elapsed >= shutdownDelay) {
    digitalWrite(motorPin, LOW);
    digitalWrite(ledPin, LOW);
    digitalWrite(buzzerPin, LOW);
    motorRunning = false;
    shutdownInitiated = false;
    Serial.println("Motor OFF (Unsafe expired)");
    display.setCursor(0, 35);
    display.println("Motor OFF (Unsafe)");
}
}
}
// === MOTOR OFF ===
else {
    digitalWrite(motorPin, LOW);
    digitalWrite(ledPin, LOW);
    digitalWrite(buzzerPin, LOW);
    display.setCursor(0, 35);
    display.println("Motor: OFF");
}
display.display();
delay(300);
}

```

7.2 Sample Screenshots

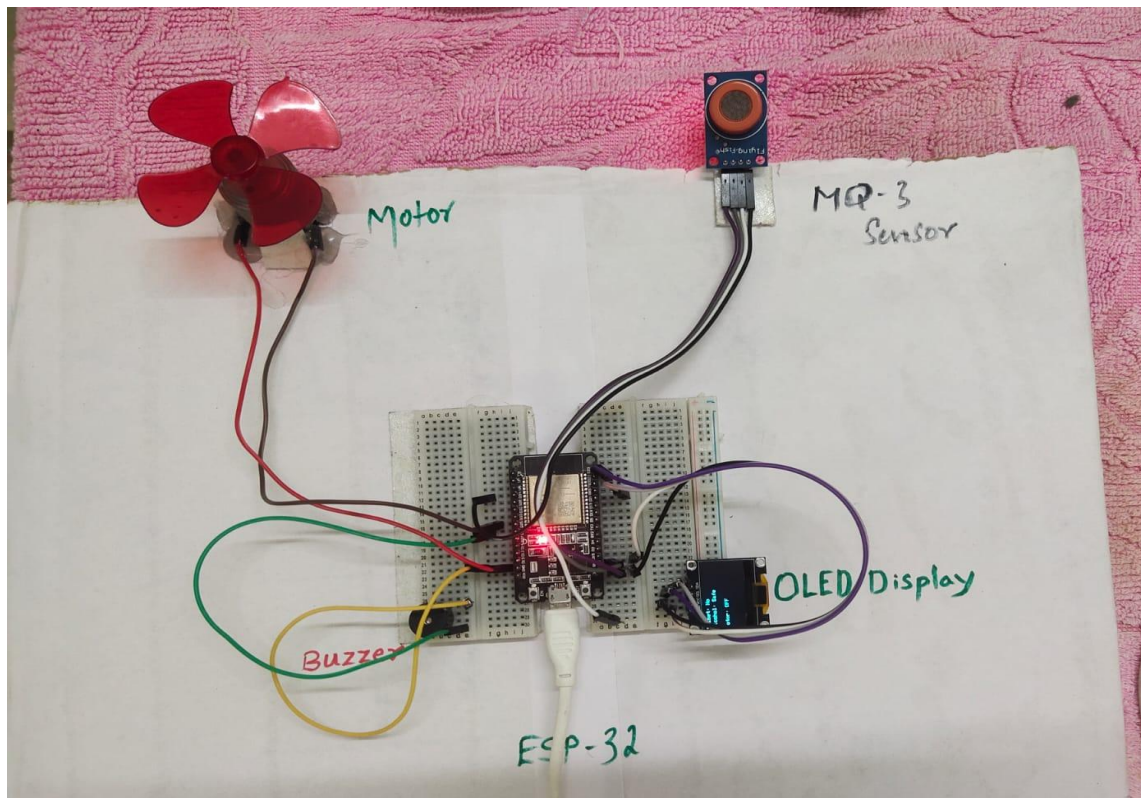


Figure 5: Hardware setup

Fig 5: Shows the actual physical setup including ESP32, MQ-3, OLED, motor and breadboard wiring. This represents the real prototype used for testing.

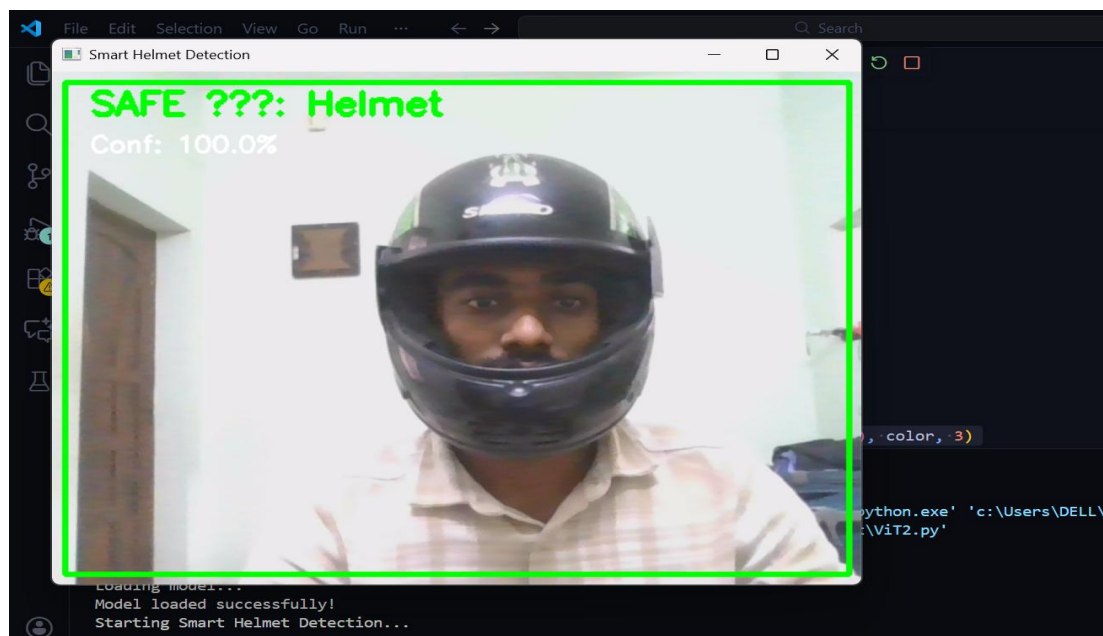


Figure 6: Helmet detected

Fig 6: Displays system output when a helmet is detected. Motor remains ON because safety condition is satisfied.

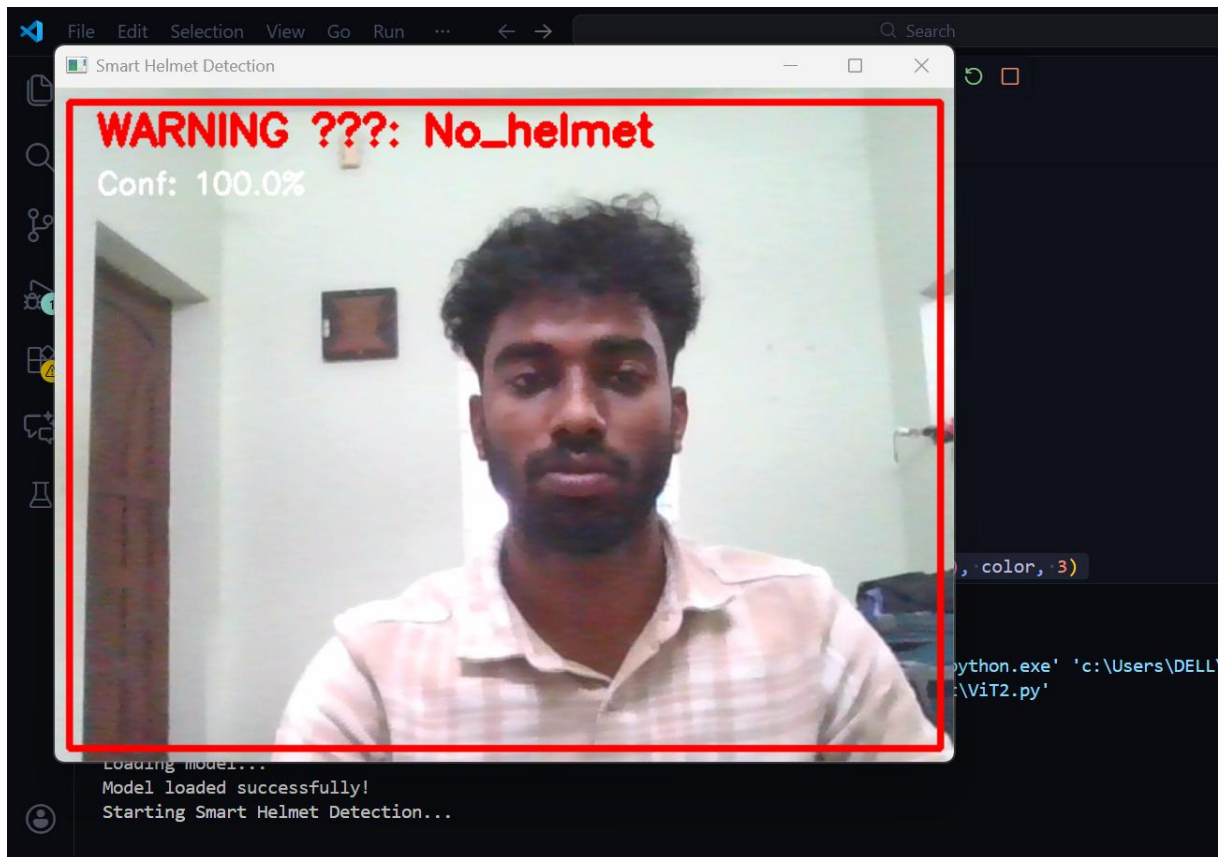


Figure 7: NO helmet detected

Fig 7: Shows the alert given when no helmet is detected. Ignition is disabled to prevent unsafe riding.

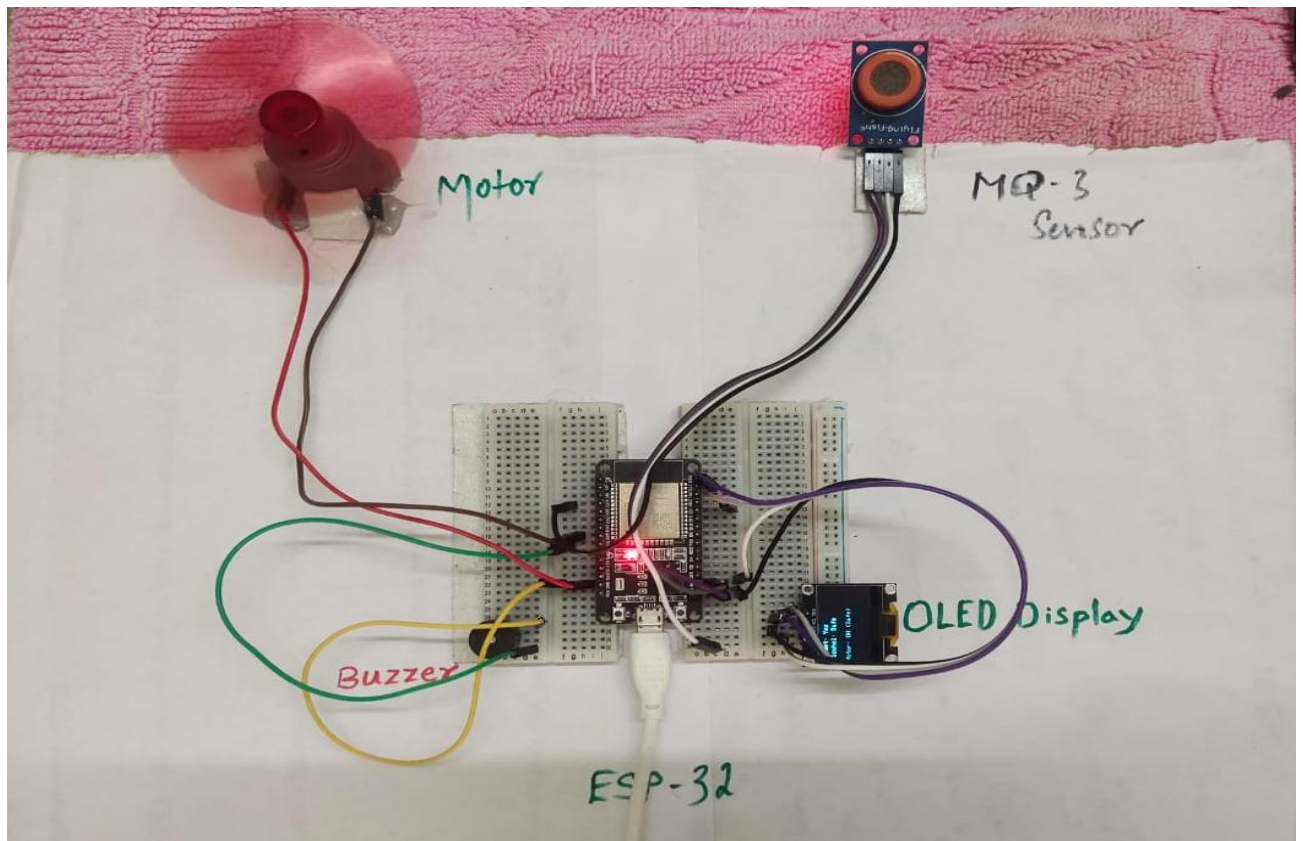


Figure 8: If helmet detected and alcohol is safe the motor will ON

Fig 8: Both conditions are safe, so the system allows the motor to run. This is the only state where the engine remains ON.



Figure 9: NO helmet and Alcohol Safe

Fig 9: Helmet is missing but there is no alcohol detected. System still blocks ignition because helmet condition is mandatory.



Figure 10: If Alcohol Detected

Fig 10: Sensor detects alcohol and flags the rider as unsafe. Motor remains OFF even if helmet is present.



Figure 11: If both are True

Fig 11: Helmet is worn and no alcohol is detected, both are valid. System allows normal safe operation.



Figure 12: In between riding anything went false the countdown of 60sec
 Fig 12: Triggers when riding becomes unsafe after the motor is already running. Provides 60 seconds for the rider to correct the issue before shutdown.

Metric Table 1&2:

Metric	Formula	Value	Percentage
Accuracy	$TP+TN/TP+TN+FP+FN$	0.9988	99.88%
Precision	$TP/TP+FP$	0.9988	99.88%
Recall	$TP/TP+FN$	0.9988	99.88%
F1-Score	$2 \times (Precision \times Recall) / (Precision + Recall)$	0.9988	99.88%

Class	Precision	Recall	F1-Score	Support
Helmet	0.9976	1.0000	0.9988	409.0000
No_helmet	1.0000	0.9977	0.9988	431.0000
Accuracy	0.9988	0.9988	0.9988	0.9988
Macro Avg	0.9988	0.9988	0.9988	840.0000
Weighted Avg	0.9988	0.9988	0.9988	840.0000

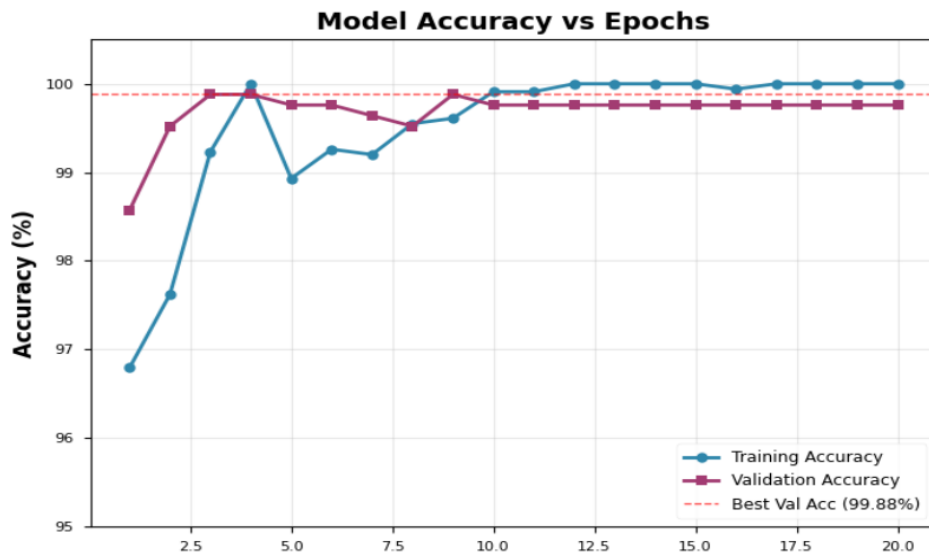


Figure 13: Model Accuracy vs Epochs

Fig 13: Shows how accuracy improves during training and validation. Demonstrates convergence and good performance.

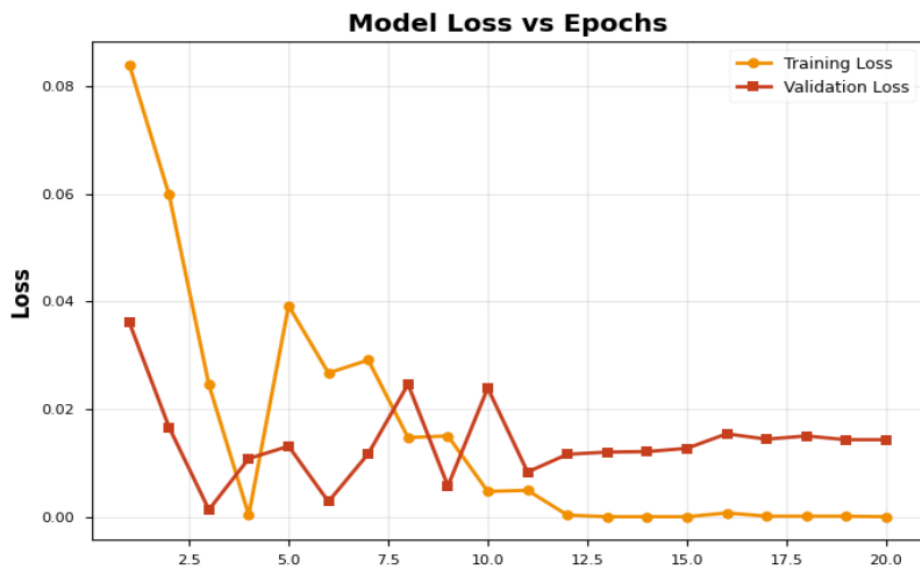


Figure 14: Model Loss vs Epochs

Fig 14: Displays loss value decreasing over time. Lower loss indicates effective learning.

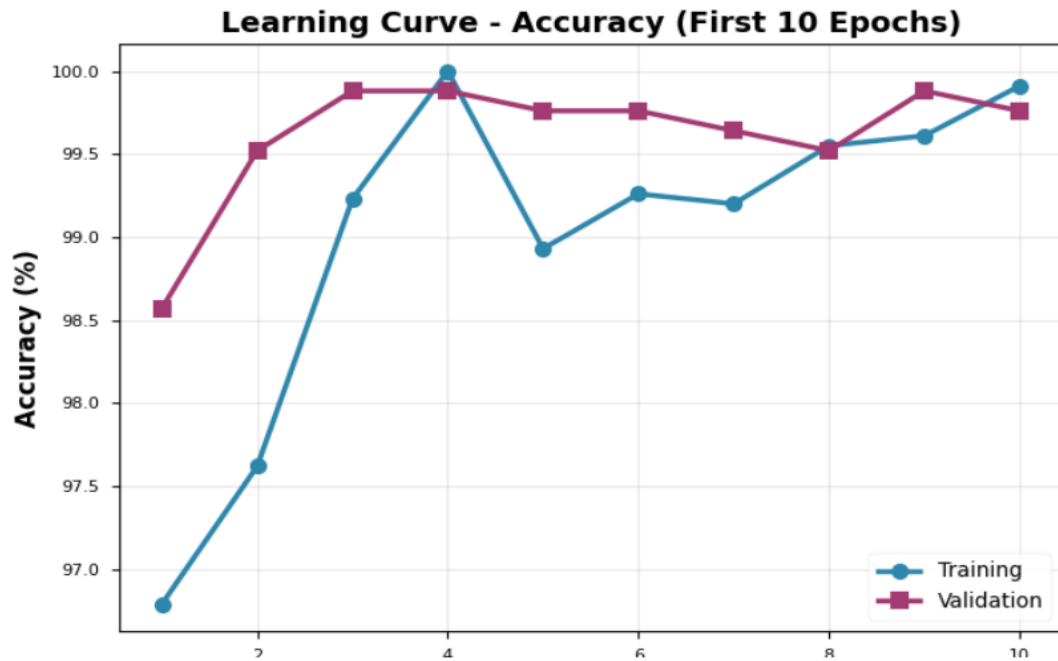


Figure 15: Learning cure Accuracy

Fig 15: Another view of how accuracy evolves over epochs. Stable high accuracy indicates reliable model behavior.

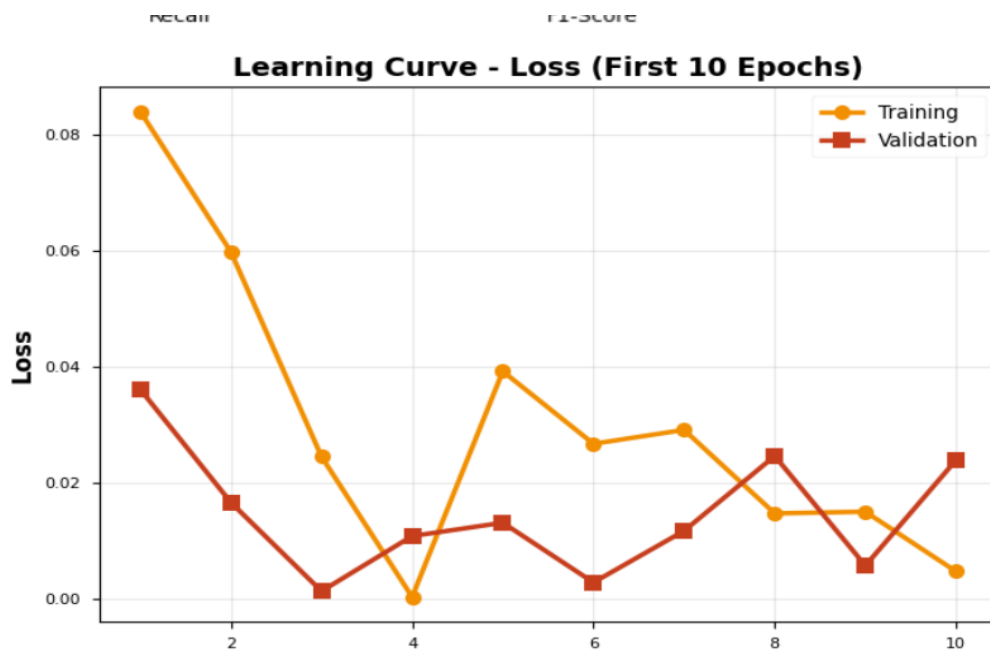


Figure 16: Learning curve Loss

Fig 16: Shows the reduction in training and validation loss. Smooth curve indicates no overfitting and balanced training.

CHAPTER 8

COST ANALYSIS

Component	Quantity	Updated Cost (INR)
ESP32 Development Board	1	₹450
MQ-3 Alcohol Sensor	1	₹200
DC Motor	1	₹50
OLED Display (0.96" I2C)	1	₹160
Breadboard	2	₹200
Jumper Wires (Male–Male / Male–Female)	1 set	₹150
USB Cable (for ESP32)	1	₹80
Buzzer (Active 5V)	1	₹40
Webcam (USB)	1	₹700
Total Estimated Cost	—	₹2,030

RESULT & DISCUSSION

The proposed hybrid safety enforcement system integrating a Vision Transformer–CNN helmet detection model with an ESP32-based alcohol detection module demonstrates high accuracy, reliability, and real-time performance. The helmet detection model was trained using a custom dataset comprising Helmet and No Helmet classes with extensive augmentation (rotation, blur, brightness, horizontal flip) to ensure robustness against real-world variations. Training and validation accuracy curves show smooth convergence with minimal overfitting, achieving a peak validation accuracy of 99.88%, demonstrating the effectiveness of the integrated ViT + CNN feature fusion approach. The confusion matrix, precision, recall, and F1-score confirm that the model reliably classifies both helmeted and non-helmeted riders under varying lighting and orientation conditions.

The MQ-3 alcohol sensor was interfaced with an ESP32 microcontroller to detect ethanol content in the rider's breath. Real-time analog voltage readings were converted into threshold-based alcohol status, classified as SAFE or UNSAFE. The integration logic ensures that ignition is enabled only when Helmet = Detected AND Alcohol = Safe. This hardware-software coupling allows direct enforcement instead of passive monitoring.

A 60-second safety shutdown mechanism was implemented to prevent abrupt motor cut-off

during motion. When unsafe behavior is detected (helmet removed / alcohol detected), a countdown timer is displayed on the OLED screen and buzzer warnings are triggered every 5 seconds. If the rider meets the safety condition within this interval, the shutdown is canceled; otherwise, the motor relay is disabled automatically.

Live detection testing confirms that the system responds instantly to camera input and sensor readings, with an average inference time under 100ms, making it suitable for real-time deployment. Experimental results show that the hybrid AI-IoT framework provides proactive accident prevention rather than just post-event reporting.

Overall, the results validate that combining deep learning-based helmet detection with alcohol sensing and ignition control effectively reduces the possibility of riding under unsafe conditions. The system demonstrates high accuracy, low latency, and practical feasibility for real-world two-wheeler safety applications.

CHAPTER 9

SUMMARY

This project successfully demonstrates an integrated deep learning and embedded system solution for real-time rider safety through automated helmet compliance and alcohol detection. A hybrid Vision Transformer + CNN fusion model was developed to classify helmet usage from live camera input with high accuracy. In parallel, the MQ-3 alcohol sensor interfaced with ESP32 continuously monitored breath alcohol levels. A safety control logic enforced ignition only when both conditions—helmet present and safe alcohol level—were satisfied.

A complete system pipeline was implemented, including dataset collection, augmentation, model training, validation, hardware interfacing, serial communication, relay control, and OLED-based feedback. The system achieved 99.88% accuracy in helmet recognition and demonstrated reliable alcohol detection through analog thresholding. The 60-second controlled shutdown mechanism further improved usability by preventing abrupt motor off conditions while still enforcing safety.

Through testing, the system proved effective at detecting unsafe scenarios (no helmet or alcohol detected) and successfully responded through visual (OLED), auditory (buzzer), and mechanical (relay cutoff) feedback. These results validate its potential for real-world applications in motorcycle safety, public transport, and fleet management.

Expected Results

The system is expected to provide:

- High accuracy helmet detection using ViT-based global attention and CNN spatial features
- Reliable alcohol detection via MQ-3 sensor with real-time threshold evaluation
- Automatic ignition lock and smart shutdown event handling
- Real-time visual and audio feedback using OLED and buzzer
- A scalable architecture that can be extended with GPS, GSM, cloud logging, or mobile apps

Recommendations

- Train the model with additional datasets to improve performance across various riding environments
- Deploy model optimization (ONNX, TensorRT, quantization) for edge deployment on ESP32-S3 or Jetson Nano
- Add cloud integration for vehicle tracking, alert logging, and owner notifications
- Integrate GPS/IoT modules for remote monitoring and law enforcement support
- Expand the system to detect additional safety violations such as multiple riders, overspeeding, or accident detection

CHAPTER 10

REFERENCES

1. Chen, S., Sun, H., Wu, Y., Shang, L., & Ruan, X. (2025). A helmet detection algorithm based on transformers with deformable attention module. *Chinese Journal of Electronics*.
2. Vatkar, N. S., et al. (2024, November). Alcohol detection and engine lock system. *International Research Journal of Modernization in Engineering Technology and Science*
3. Anil, M. A., et al. (2025). Smart helmet — alcohol detection to avoid accidents. *Journal of Aerospace and Aeronautical Research (JAAFR)*.
4. Alabi, O. O., et al. (2024, July). Design and implementation of an alcohol detection driver system. *International Journal of Research in Engineering and Science (IJRES)*.
5. Yasin, M., et al. (2025). AI-driven automated helmet detection in underground coal mines using attention-enhanced vision transformer. *Engineering, Technology & Applied Science Research (ETASR)*.
6. Su, X., Chen, Y., Fang, L., & Zhang, W. (2023). A novel transformer-based adaptive object detection method. *Electronics*, 12(15), 3298.
7. Ye, Y., et al. (2025). A lightweight mechanism for vision-transformer-based object detection (XFCOS/XFA). *Complex & Intelligent Systems*.
8. Yogeshwaran, K., Dharshini, V. G., Aishwarya, S., & Elanthamizan, A. (2025). Advanced smart safety helmet for motor cycles using real-time sensor data. *International Journal of Engineering Research and Technology*.
9. Chen, S., Sun, H., Wu, Y., Shang, L., & Ruan, X. (2025). A helmet detection algorithm based on transformers with deformable attention module. *IEEE Transactions on Industrial Electronics*.
10. Zhang, L., & Zhao, M. (2023). Helmet detection using YOLOv5 and DeepSort. *Computer Vision and Pattern Recognition*.
11. Kumaravel, S., & Ramesh, K. (2022). Motorcyclist helmet detection using CNNs. *International Journal of Computer Applications*.
12. Ahmed, A., & Khan, M. (2021). Smart helmet for accident detection and prevention. *IEEE Internet of Things Journal*.
13. Priya, R., & Manjunath, H. (2020). A review of deep learning techniques for helmet detection. *Machine Learning and Applications*.

14. Wang, Y., & Liu, T. (2019). Automatic helmet detection in real-time surveillance videos. *Computer Vision and Image Understanding*.

15. Kumar, K. K., Korlam, S. S. V. P., Kumar, K. N., & Prasad, B. D. V. (2023). Harnessing IoT for road safety: Intelligent helmets with ignition shut-off. *Internet of Things and Applications*.